

From Kubernetes Security to Policy as Code

- We have secured the cluster using Kubernetes' native security features.
- Lab 5 introduced:
 - RBAC – controls who can perform actions
 - NetworkPolicies – control workload communication
 - Pod Security Admission – controls how Pods are allowed to run
- The next question is: how do we enforce these standards consistently across every application?

The Challenge

- Every application team must remember to:
 - Add securityContext
 - Specify CPU and memory requests/limits
 - Use approved container images
 - Apply labels and naming standards
- If any requirement is missed, deployments fail.
- Policy as Code automates these checks and keeps standards consistent.

Policy as Code with Kyverno

- Kyverno extends Kubernetes with organisation-specific policy enforcement.
- Kubernetes validates resources, but not your company's standards.
- Examples:
 - Approved container images
 - Mandatory labels
 - CPU and memory limits
 - Non-root containers
 - Automatic ResourceQuotas

Where Does Kyverno Fit?

- Developer runs: `kubectl apply`
- API Server receives the request
- Admission Controllers evaluate the request
- Kyverno validates or mutates the resource
- Only compliant resources are stored in etcd

Why Do We Need Kyverno?

- Manual governance does not scale.
- Developers forget security settings.
- Platform teams need consistent enforcement.
- Write the policy once; enforce it everywhere.

The Four Kyverno Policy Types

- Validate – reject non-compliant resources
- Mutate – modify resources automatically
- Generate – create supporting resources
- Cleanup – remove obsolete resources

Validate Policies

- Typical examples:
 - Reject nginx:latest
 - Require securityContext
 - Require labels
 - Restrict image registries
- Lab 6b introduces your first validation policy.

Mutate Policies

- Automatically add:
 - Labels
 - Annotations
 - Image pull secrets
 - Tolerations
- No changes required from the developer.

Generate and Cleanup

- Generate:
 - ResourceQuotas
 - LimitRanges
 - NetworkPolicies
- Cleanup:
 - Completed Jobs
 - Temporary resources
 - Expired objects

How Lab 6 Fits Together

- Lab 6a – Build a governed Kubernetes platform
- Lab 6b – Create your first Kyverno validation policy
- Lab 6c – Explore enterprise Policy as Code
- Goal: move from manual governance to automated governance.

Lab 6 Roadmap

- Lab 6a – Build a governed Kubernetes platform
 - Namespaces
 - Pod Security
 - ResourceQuotas
 - NetworkPolicies
 - Ingress
- Lab 6b – Introduce Kyverno validation policies
- Lab 6c – Explore Validate, Mutate, Generate and Cleanup policies

Changing Roles Throughout Lab 6

- 6a.1 – Platform Engineer
 - Build the platform and apply governance
- 6a.2 – Application Developer
 - Deploy an application and resolve policy failures
- 6a.3 – Platform Troubleshooter
 - Publish the application and diagnose networking issues
- 6b/6c – Platform Administrator
 - Automate governance using Kyverno

The Learning Journey

- Manual Security Controls (Lab 5)
 - RBAC
 - NetworkPolicies
 - Pod Security
- Governed Platform (Lab 6a)
- Policy as Code (Lab 6b)
- Enterprise Automation (Lab 6c)